Paradigmas de Programación [1]

Unificación
Inferencia de tipos [2]

2do cuatrimestre de 2025 [3]

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Introducción 1

Algoritmo de unificación 2

Algoritmo de inferencia de tipos 3

Corrección del algoritmo de unificación 5

Problema de inferencia de tipos 1

Notación 2

Términos **sin** anotaciones de tipos: 3

$$U ::= x \mid \lambda x.\, U \mid U\, U \mid \mathsf{True} \mid \mathsf{False} \mid \mathsf{if}\ U\ \mathsf{then}\ U\ \mathsf{else}\ U$$ 4

Términos **con** anotaciones de tipos: 5

$$M ::= x \mid \lambda x : \tau.\, M \mid M\, M \mid \mathsf{True} \mid \mathsf{False} \mid \mathsf{if}\ M\ \mathsf{then}\ M\ \mathsf{else}\ M$$ 6

Notamos $\mathtt{erase}(M)$ al término sin anotaciones de tipos que resulta de borrar las anotaciones de tipos de M. 7

Ejemplo: $\mathtt{erase}((\lambda x : \mathsf{Bool}.\, x)\, \mathsf{True}) = (\lambda x.\, x)\, \mathsf{True}$. 8

Problema de inferencia de tipos 1

Definición 2

Un término U sin anotaciones de tipos es **tipable** sii existen: 3

un contexto de tipado Γ 4

un término con anotaciones de tipos M

un tipo τ

tales que $\texttt{erase}(M) = U$ y $\Gamma \vdash M : \tau$.

El **problema de inferencia de tipos** consiste en: 5

- Dado un término U, determinar si es tipable. 6
- En caso de que U sea tipable:
  hallar un contexto Γ, un término M y un tipo τ
  tales que $\texttt{erase}(M) = U$ y $\Gamma \vdash M : \tau$.

Veremos un algoritmo para resolver este problema. 7

Problema de inferencia de tipos 1

El algoritmo se basa en manipular tipos *parcialmente conocidos*. 2

Ejemplo — tipos parcialmente conocidos 3

- En x True sabemos que $x : \text{Bool} \to \mathtt{X}_1$. 4
- En if x y then True else False sabemos que $x : \mathtt{X}_2 \to \text{Bool}$.

Incorporamos *incógnitas* $(\mathtt{X}_1, \mathtt{X}_2, \mathtt{X}_3, \ldots)$ a los tipos. 5

Vamos a necesitar resolver *ecuaciones* entre tipos con incógnitas. 6

Ejemplo — ecuaciones entre tipos 7

- $(\mathtt{X}_1 \to \text{Bool}) \stackrel{?}{=} ((\text{Bool} \to \text{Bool}) \to \mathtt{X}_2)$ 8
  tiene solución: $\mathtt{X}_1 := (\text{Bool} \to \text{Bool})$ y $\mathtt{X}_2 := \text{Bool}$.
- $(\mathtt{X}_1 \to \mathtt{X}_1) \stackrel{?}{=} ((\text{Bool} \to \text{Bool}) \to \mathtt{X}_2)$ 9
  tiene solución: $\mathtt{X}_1 := (\text{Bool} \to \text{Bool})$ y $\mathtt{X}_2 := (\text{Bool} \to \text{Bool})$.
- $(\mathtt{X}_1 \to \text{Bool}) \stackrel{?}{=} \mathtt{X}_1$
  no tiene solución.

Introducción 1

Algoritmo de unificación 2

Algoritmo de inferencia de tipos 3

Corrección del algoritmo de unificación 4

Unificación 1

Suponemos fijado un conjunto finito de constructores de tipos: 2

- Tipos constantes: Bool, Int, ...
- Constructores unarios: (List •), (Maybe •), ...
- Constructores binarios: (• → •), (• × •), (Either • •), ...
- (Etcétera).

3

Los tipos se forman usando incógnitas y constructores: 4

$$\tau ::= \mathtt{X}_n \mid C(\tau_1, \ldots, \tau_n)$$ 5

La **unificación** es el problema de resolver sistemas de ecuaciones entre tipos con incógnitas. 6

Veremos primero un algoritmo de unificación.
Luego lo usaremos para dar un algoritmo de inferencia de tipos. 7

Unificación 1

Una **sustitución** es una función que a cada incógnita le asocia un tipo. 2

Notamos: 3

$$\{\mathtt{X}_{k_1} := \tau_1, \ldots, \mathtt{X}_{k_n} := \tau_n\}$$ 4

a la sustitución $\mathbf{S}$ tal que $\mathbf{S}(\mathtt{X}_{k_i}) = \tau_i$ para cada $1 \leq i \leq n$ y $\mathbf{S}(\mathtt{X}_k) = \mathtt{X}_k$ para cualquier otra incógnita. 5

Si τ es un tipo, escribimos $\mathbf{S}(\tau)$ para el resultado de reemplazar cada incógnita de τ por el valor que le otorga $\mathbf{S}$. 6

Ejemplo — aplicación de una sustitución a un tipo 7

Si $\mathbf{S} = \{\mathtt{X}_1 := \mathrm{Bool},\ \mathtt{X}_3 := (\mathtt{X}_2 \to \mathtt{X}_2)\}$, entonces: 8

$$\mathbf{S}((\mathtt{X}_1 \to \mathrm{Bool}) \to \mathtt{X}_3) = ((\mathrm{Bool} \to \mathrm{Bool}) \to (\mathtt{X}_2 \to \mathtt{X}_2))$$ 9

Unificación 1

Un **problema de unificación** es un conjunto finito E de ecuaciones entre tipos que pueden involucrar incógnitas: 2

$$E = \{\tau_1 \stackrel{?}{=} \sigma_1, \tau_2 \stackrel{?}{=} \sigma_2, \ldots, \tau_n \stackrel{?}{=} \sigma_n\}$$ 3

Un **unificador** para E es una sustitución $\mathbf{S}$ tal que: 5

$$\mathbf{S}(\tau_1) = \mathbf{S}(\sigma_1)$$ 6

$$\mathbf{S}(\tau_2) = \mathbf{S}(\sigma_2)$$

$$\ldots$$

$$\mathbf{S}(\tau_n) = \mathbf{S}(\sigma_n)$$ 7

Unificación

En general, la solución a un problema de unificación no es única. 1

Ejemplo — problema de unificación con infinitas soluciones 2

$$\{\mathtt{X}_1 \stackrel{?}{=} \mathtt{X}_2\}$$ 3

tiene infinitos unificadores: 4

- $\{\mathtt{X}_1 := \mathtt{X}_2\}$
- $\{\mathtt{X}_2 := \mathtt{X}_1\}$
- $\{\mathtt{X}_1 := \mathtt{X}_3,\ \mathtt{X}_2 := \mathtt{X}_3\}$
- $\{\mathtt{X}_1 := \mathsf{Bool},\ \mathtt{X}_2 := \mathsf{Bool}\}$
- $\{\mathtt{X}_1 := (\mathsf{Bool} \to \mathsf{Bool}),\ \mathtt{X}_2 := (\mathsf{Bool} \to \mathsf{Bool})\}$
- ...

5

Unificación 1

Una sustitución $\mathbf{S}_A$ es **más general** que una sustitución $\mathbf{S}_B$ si existe una sustitución $\mathbf{S}_C$ tal que: 2

$$\mathbf{S}_B = \mathbf{S}_C \circ \mathbf{S}_A$$ 3

es decir, $\mathbf{S}_B$ se obtiene instanciando variables de $\mathbf{S}_A$. 4

Para el siguiente problema de unificación: 5

$$E = \{(\mathtt{X}_1 \to \mathsf{Bool}) \stackrel{?}{=} \mathtt{X}_2\}$$ 6

las siguientes sustituciones son unificadores: 7

- $\mathbf{S}_1 = \{\mathtt{X}_1 := \mathsf{Bool},\ \mathtt{X}_2 := (\mathsf{Bool} \to \mathsf{Bool})\}$ 8
- $\mathbf{S}_2 = \{\mathtt{X}_1 := \mathsf{Int},\ \mathtt{X}_2 := (\mathsf{Int} \to \mathsf{Bool})\}$
- $\mathbf{S}_3 = \{\mathtt{X}_1 := \mathtt{X}_3,\ \mathtt{X}_2 := (\mathtt{X}_3 \to \mathsf{Bool})\}$
- $\mathbf{S}_4 = \{\mathtt{X}_2 := (\mathtt{X}_1 \to \mathsf{Bool})\}$

¿Qué relación hay entre ellas? (¿Cuál es más general que cuál?). 9

Algoritmo de unificación de Martelli–Montanari 1

Dado un problema de unificación E (conjunto de ecuaciones): 2

- Mientras $E \neq \varnothing$, se aplica sucesivamente alguna de las seis reglas que se detallan más adelante. 3
- La regla puede resultar en una `falla`.
- De lo contrario, la regla es de la forma $E \rightarrow_{\mathbf{S}} E'$.
  La resolución del problema E se reduce a resolver otro problema E', aplicando la sustitución $\mathbf{S}$.

Hay dos posibilidades: 4

1. $E = E_0 \rightarrow_{\mathbf{S}_1} E_1 \rightarrow_{\mathbf{S}_2} E_2 \rightarrow \ldots \rightarrow_{\mathbf{S}_n} E_n \rightarrow_{\mathbf{S}_{n+1}} \texttt{falla}$ 5
   En tal caso el problema de unificación E no tiene solución.
2. $E = E_0 \rightarrow_{\mathbf{S}_1} E_1 \rightarrow_{\mathbf{S}_2} E_2 \rightarrow \ldots \rightarrow_{\mathbf{S}_n} E_n = \varnothing$
   En tal caso el problema de unificación E tiene solución.

Algoritmo de unificación de Martelli–Montanari 1

2

$$\{\mathtt{X}_n \stackrel{?}{=} \mathtt{X}_n\} \cup E \xrightarrow{\texttt{Delete}} E$$

$$\{C(\tau_1, \ldots, \tau_n) \stackrel{?}{=} C(\sigma_1, \ldots, \sigma_n)\} \cup E \xrightarrow{\texttt{Decompose}} \{\tau_1 \stackrel{?}{=} \sigma_1, \ldots, \tau_n \stackrel{?}{=} \sigma_n\} \cup E$$

4

$$\{\tau \stackrel{?}{=} \mathtt{X}_n\} \cup E \xrightarrow{\texttt{Swap}} \{\mathtt{X}_n \stackrel{?}{=} \tau\} \cup E$$

si τ no es una incógnita

$$\{\mathtt{X}_n \stackrel{?}{=} \tau\} \cup E \xrightarrow{\texttt{Elim}}_{\{\mathtt{X}_n := \tau\}} E' = \{\mathtt{X}_n := \tau\}(E)$$

si $\mathtt{X}_n$ no ocurre en τ

$$\{C(\tau_1, \ldots, \tau_n) \stackrel{?}{=} C'(\sigma_1, \ldots, \sigma_m)\} \cup E \xrightarrow{\texttt{Clash}} \texttt{falla}$$

si $C \neq C'$

$$\{\mathtt{X}_n \stackrel{?}{=} \tau\} \cup E \xrightarrow{\texttt{Occurs-Check}} \texttt{falla}$$

si $\mathtt{X}_n \neq \tau$
y $\mathtt{X}_n$ ocurre en τ

Algoritmo de unificación de Martelli–Montanari 1

Teorema (Corrección del algoritmo de Martelli–Montanari) 2

1. El algoritmo termina para cualquier problema de unificación E. 3
2. Si E no tiene solución, el algoritmo llega a una `falla`. 4
3. Si E tiene solución, el algoritmo llega a $\varnothing$: 5

$$E = E_0 \rightarrow_{\mathbf{S}_1} E_1 \rightarrow_{\mathbf{S}_2} E_2 \rightarrow \ldots \rightarrow_{\mathbf{S}_n} E_n = \varnothing \quad 6$$

Además, $\mathbf{S} = \mathbf{S}_n \circ \ldots \circ \mathbf{S}_2 \circ \mathbf{S}_1$ es un unificador para E. 7

Además, dicho unificador es el *más general* posible. 8
(Salvo renombre de incógnitas).

Definición (Unificador más general) 9

Notamos $\mathrm{mgu}(E)$ al unificador más general de E, si existe. 10

Algoritmo de unificación de Martelli–Montanari [1]

Ejemplo [2]

Calcular unificadores más generales para los siguientes problemas [3] de unificación:

- $\{(\mathtt{X}_2 \to (\mathtt{X}_1 \to \mathtt{X}_1)) \stackrel{?}{=} ((\mathsf{Bool} \to \mathsf{Bool}) \to (\mathtt{X}_1 \to \mathtt{X}_2))\}$ [4]
- $\{\mathtt{X}_1 \stackrel{?}{=} (\mathtt{X}_2 \to \mathtt{X}_2),\ \mathtt{X}_2 \stackrel{?}{=} (\mathtt{X}_1 \to \mathtt{X}_1)\}$

Introducción 1

Algoritmo de unificación 2

Algoritmo de inferencia de tipos 3

Corrección del algoritmo de unificación 4

Algoritmo $\mathcal{I}$ — Inferencia de tipos 1

El algoritmo $\mathcal{I}$ recibe un término U sin anotaciones de tipos. 2

Consta de los siguientes pasos: 3

1. **Rectificación** del término. 4
2. **Anotación** del término con variables de tipo frescas.
3. **Generación de restricciones** (ecuaciones entre tipos).
4. **Unificación de las restricciones**.

Algoritmo $\mathcal{I}$ — Paso 1: rectificación 1

Decimos que un término está *rectificado* si: 2

1. No hay dos variables ligadas con el mismo nombre. 3
2. No hay una variable ligada con el mismo nombre que una variable libre.

Ejemplo – Términos rectificados 4

$x\,(\lambda x.\, x\, x)\,(\lambda y.\, y\, x)$	no está rectificado 5
$x\,(\lambda z.\, z\, z)\,(\lambda y.\, y\, x)$	está rectificado
$\lambda x.\, \lambda x.\, x\, y$	no está rectificado
$\lambda x.\, \lambda z.\, z\, y$	está rectificado

Observación 6

Siempre se puede rectificar un término α-renombrándolo. 7

Algoritmo $\mathcal{I}$ — Paso 2: anotación 1

Tenemos un término U, que suponemos ya rectificado. 2

Producimos un contexto Γ_0 y un término M_0: 3

1. El contexto Γ_0 le da tipo a todas las variables libres de U. 4
   El tipo de cada variable es una incógnita *fresca*.
2. El término M_0 está anotado de tal modo que $\texttt{erase}(M_0) = U$.
   Todas las anotaciones son incógnitas frescas.

Ejemplo – Anotación del término 5

Dado el término rectificado $U = (\lambda x.\, y\, x\, x)\,(\lambda z.\, w)$, producimos: 6

1. $\Gamma_0 = (y : \mathtt{X}_1, w : \mathtt{X}_2)$ 7
2. $M_0 = (\lambda x : \mathtt{X}_3.\, y\, x\, x)\,(\lambda z : \mathtt{X}_4.\, w)$

Algoritmo $\mathcal{I}$ — Paso 3: generación de las restricciones 1

Tenemos un contexto Γ y un término M con anotaciones de tipos. 2

Recursivamente calculamos: 3

1. Un tipo τ, que corresponde al tipo de M. 4
2. Un conjunto de ecuaciones E.
   Representan restricciones para que M esté bien tipado.

Definimos un algoritmo recursivo: 5

$$\mathcal{I}(\underbrace{\underbrace{\Gamma}_{\text{contexto}} \mid \underbrace{M}_{\text{término}}}_{\text{entrada}}) = (\underbrace{\underbrace{\tau}_{\text{tipo}} \mid \underbrace{E}_{\text{restricciones}}}_{\text{salida}})$$ 6

con la precondición de que Γ le da tipo a todas las variables de M. 7

Algoritmo $\mathcal{I}$ — Paso 3: generación de las restricciones 1

1. $\mathcal{I}(\Gamma \mid \text{True}) = (\text{Bool} \mid \varnothing)$
2. $\mathcal{I}(\Gamma \mid \text{False}) = (\text{Bool} \mid \varnothing)$
3. $\mathcal{I}(\Gamma \mid x) = (\tau \mid \varnothing) \qquad \text{si } (x : \tau) \in \Gamma$
4. $\mathcal{I}(\Gamma \mid \text{if } M_1 \text{ then } M_2 \text{ else } M_3) =$
$$(\tau_2 \mid \{\tau_1 \stackrel{?}{=} \text{Bool}, \tau_2 \stackrel{?}{=} \tau_3\} \cup E_1 \cup E_2 \cup E_3)$$
donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
$\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$
$\mathcal{I}(\Gamma \mid M_3) = (\tau_3 \mid E_3)$

2

5. $\mathcal{I}(\Gamma \mid M_1\, M_2) = (\mathtt{X}_k \mid \{\tau_1 \stackrel{?}{=} (\tau_2 \to \mathtt{X}_k)\} \cup E_1 \cup E_2)$ 3
donde $\mathtt{X}_k$ es una incógnita fresca 4
$\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
$\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$
6. $\mathcal{I}(\Gamma \mid \lambda x : \tau.\ M) = (\tau \to \sigma \mid E)$ 5
donde $\mathcal{I}(\Gamma, x : \tau \mid M) = (\sigma \mid E)$

Algoritmo $\mathcal{I}$ — Paso 4: unificación de las restricciones 1

Recordemos: Γ_0 y M_0 resultan de anotar un término rectificado U. 2

Una vez calculado $\mathcal{I}(\Gamma_0 \mid M_0) = (\tau \mid E)$: 3

1. Calculamos $\mathbf{S} = \mathrm{mgu}(E)$. 4
2. Si no existe el unificador, el término U no es tipable.
3. Si existe el unificador, el término U es tipable y vale:

$$\mathbf{S}(\Gamma_0) \vdash \mathbf{S}(M_0) : \mathbf{S}(\tau)$$ 5

Algoritmo $\mathcal{I}$ — Corrección 1

Teorema (Corrección del algoritmo $\mathcal{I}$) 2

Sean Γ_0 y M_0 el resultado de anotar un término rectificado U. 3
Supongamos que $\mathcal{I}(\Gamma_0 \mid M_0) = (\tau \mid E)$. Entonces:

1. Si U no es tipable, no hay unificador para E. 4
2. Si U es tipable, existe $\mathbf{S} = \mathrm{mgu}(E)$.
   Además, $\mathbf{S}(\Gamma_0) \vdash \mathbf{S}(M_0) : \mathbf{S}(\tau)$ es un juicio de tipado válido.

   Más aún, el juicio de tipado es el más general posible para U.
   Más precisamente, si $\Gamma' \vdash M' : \tau'$ es un juicio válido y
   $\mathtt{erase}(M') = U$, existe una sustitución $\mathbf{S}'$ tal que:

$$\begin{array}{rcl} \Gamma' & \supseteq & \mathbf{S}'(\Gamma_0) \\ M' & = & \mathbf{S}'(M_0) \\ \tau' & = & \mathbf{S}'(\tau) \end{array}$$ 5

donde además $\mathbf{S}$ es más general que $\mathbf{S}'$. 6

Algoritmo $\mathcal{I}$ de inferencia de tipos [1]

Ejercicio. Aplicar el algoritmo de inferencia sobre los siguientes términos: [2]

- $\lambda x.\, \lambda y.\, y\, x$ [3]
- $(\lambda x.\, x\, x)(\lambda x.\, x\, x)$

Introducción 1

Algoritmo de unificación 2

Algoritmo de inferencia de tipos 3

Corrección del algoritmo de unificación 4

Recordemos: algoritmo de unificación 1

2 3

$$\{\mathtt{X}_n \stackrel{?}{=} \mathtt{X}_n\} \cup E \xrightarrow{\mathtt{Delete}} E$$

$$\{C(\tau_1,\ldots,\tau_n) \stackrel{?}{=} C(\sigma_1,\ldots,\sigma_n)\} \cup E \xrightarrow{\mathtt{Decompose}} \{\tau_1 \stackrel{?}{=} \sigma_1,\ldots,\tau_n \stackrel{?}{=} \sigma_n\} \cup E$$

$$\{\tau \stackrel{?}{=} \mathtt{X}_n\} \cup E \xrightarrow{\mathtt{Swap}} \{\mathtt{X}_n \stackrel{?}{=} \tau\} \cup E$$

si τ no es una incógnita

4

$$\{\mathtt{X}_n \stackrel{?}{=} \tau\} \cup E \xrightarrow{\mathtt{Elim}}_{\{\mathtt{X}_n := \tau\}} E\{\mathtt{X}_n := \tau\}$$

si $\mathtt{X}_n$ no ocurre en τ

$$\{C(\tau_1,\ldots,\tau_n) \stackrel{?}{=} C'(\sigma_1,\ldots,\sigma_m)\} \cup E \xrightarrow{\mathtt{Clash}} \mathtt{falla}$$

si $C \neq C'$

$$\{\mathtt{X}_n \stackrel{?}{=} \tau\} \cup E \xrightarrow{\mathtt{Occurs\text{-}Check}} \mathtt{falla}$$

si $\mathtt{X}_n \neq \tau$ y $\mathtt{X}_n$ ocurre en τ

Terminación del algoritmo de unificación 1

Dado un conjunto de ecuaciones de unificación E, definimos: 2

- n_1: cantidad de incógnitas distintas en E 3
- n_2: tamaño de E, calculado como $\sum_{(\tau \stackrel{?}{=} \sigma) \in E} |\tau| + |\sigma|$
- n_3: cantidad de ecuaciones de la forma $\tau \stackrel{?}{=} \mathtt{X}_n$ en E

Podemos observar que las reglas que no producen falla achican la tripla (n_1, n_2, n_3), de acuerdo con el *orden lexicográfico*: 4

	n_1	n_2	n_3
`Elim`	$>$		
`Decompose`	$=$	$>$	
`Delete`	$\geq$	$>$	
`Swap`	$=$	$=$	$>$

5

Corrección del algoritmo de unificación 1

Recordemos 2

1. Una **sustitución** es una función $\mathbf{S}$ que le asocia un tipo $\mathbf{S}(\mathtt{X}_n)$ a cada incógnita $\mathtt{X}_n$. 3
2. $\mathbf{S}$ es un **unificador** de E si para cada $(\tau \stackrel{?}{=} \sigma) \in E$ se tiene que $\mathbf{S}(\tau) = \mathbf{S}(\sigma)$. 4
3. $\mathbf{S}$ es **más general** que $\mathbf{S}'$ si existe $\mathbf{T}$ tal que $\mathbf{S}' = \mathbf{T} \circ \mathbf{S}$. 5
4. $\mathbf{S}$ es un **m.g.u.** de E si $\mathbf{S}$ es un unificador de E y para todo unificador $\mathbf{S}'$ de E se tiene que $\mathbf{S}$ es más general que $\mathbf{S}'$. 6 7

Técnicamente, nos interesan los m.g.u. **idempotentes**, 8 es decir $\mathbf{S}(\mathbf{S}(\tau)) = \mathbf{S}(\tau)$ para todo término τ.

Corrección del algoritmo de unificación 1

Lema — corrección de la regla `Delete` 2

$\mathbf{S}$ m.g.u. de $E \implies \mathbf{S}$ m.g.u. de $\{\mathtt{X}_n \stackrel{?}{=} \mathtt{X}_n\} \cup E$.

Lema — corrección de la regla `Swap` 3

$\mathbf{S}$ m.g.u. de $\{\tau \stackrel{?}{=} \sigma\} \cup E \implies \mathbf{S}$ m.g.u. de $\{\sigma \stackrel{?}{=} \tau\} \cup E$.

Lema — corrección de la regla `Decompose` 4

$\mathbf{S}$ m.g.u. de $\{\tau_1 \stackrel{?}{=} \sigma_1, \ldots, \tau_n \stackrel{?}{=} \sigma_n\} \cup E$

$\implies \mathbf{S}$ m.g.u. de $\{C(\tau_1, \ldots, \tau_n) \stackrel{?}{=} C(\sigma_1, \ldots, \sigma_n)\} \cup E$.

Lema — corrección de la regla `Elim` 5

$\mathbf{S}$ m.g.u. de $E\{\mathtt{X}_n := \tau\}$ y $\mathtt{X}_n \notin \tau$ 6

$\implies \mathbf{S} \circ \{\mathtt{X}_n := \tau\}$ m.g.u. de E.

Usar que si $\mathbf{S}(\mathtt{X}_n) = \tau$ entonces $\mathbf{S}(\sigma\{\mathtt{X}_n := \tau\}) = \mathbf{S}(\sigma)$. 7

Corrección del algoritmo de unificación 1

Probemos la corrección del algoritmo en caso de éxito. 2

Sea $E_0 \rightarrow_{\mathbf{s}_1} E_1 \rightarrow_{\mathbf{s}_n} E_2 \rightarrow \ldots \rightarrow_{\mathbf{s}_n} E_n = \varnothing$. 3
Veamos que $\mathbf{S}_n \circ \ldots \circ \mathbf{S}_1$ es un m.g.u. de E.
Por inducción en n:

1. Si $n = 0$, la sustitución identidad es un m.g.u. de $\varnothing$. 4
2. Si $n > 0$, se tiene: 5

$$E_0 \rightarrow_{\mathbf{s}_1} E_1 \qquad E_1 \rightarrow_{\mathbf{s}_2} \ldots \rightarrow_{\mathbf{s}_n} E_n = \varnothing$$ 6

Por HI, $\mathbf{S}_n \circ \ldots \circ \mathbf{S}_2$ es un m.g.u. de E_1. 7
Aplicando alguno de los lemas anteriores, se concluye que 8
$\mathbf{S}_n \circ \ldots \circ \mathbf{S}_2 \circ \mathbf{S}_1$ es un m.g.u. de E_0.

Corrección del algoritmo de unificación 1

La corrección en caso de falla se prueba de manera similar, con lemas que van "hacia adelante" en lugar de "hacia atrás". 2

¿ ¿ ¿ ¿ ¿ ¿ ¿ ¿ ¿ ? ? ? ? ? ? ? ? ? 1

Lectura recomendada 2

Capítulo 22 del libro de Pierce. 3

Benjamin C. Pierce. *Types and Programming Languages*. The MIT Press, 2002.

Extra: teoría detrás del método de unificación 4

Sección 4.5 del libro de Baader & Nipkow.

Franz Baader y Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.